

Chapter 3: Server-Side Web Programming

Syllabus Coverage: 9 Hours | Marks: ~15–18 **Topics:** MVC Architecture, Backend Role, Django, Requests/Responses, Forms/Sessions, Routing/Middleware/Templates, Framework Comparison, Database/ORM, Authentication, Security Middleware

3.1 Introduction to MVC Architecture

MVC (Model–View–Controller) is a software design pattern used to separate an application into three interconnected parts, making it easier to develop, maintain, test, and scale — especially in web applications.

The Three Components

Model

- Represents the **data layer and business logic**
- Defines the structure of data to be stored in the database
- Performs **CRUD operations** on the database
- Handles data and business logic, interacts with the database
- Examples: Database schemas, ORM models (Sequelize, Prisma, Mongoose, Django Models)

```
# Django Model Example
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
```

View

- Responsible for **presenting data to the user**
- Handles UI/layout/visual representation
- Shows data received from the Controller
- No business logic, no database logic**

Type	Description
Traditional MVC View (Server-Rendered)	Backend renders HTML — templates (HTML, EJS, Pug, Jinja, Blade)
Modern Decoupled View	Frontend (React/Vue/Angular) acts as the View, backend serves APIs

Controller

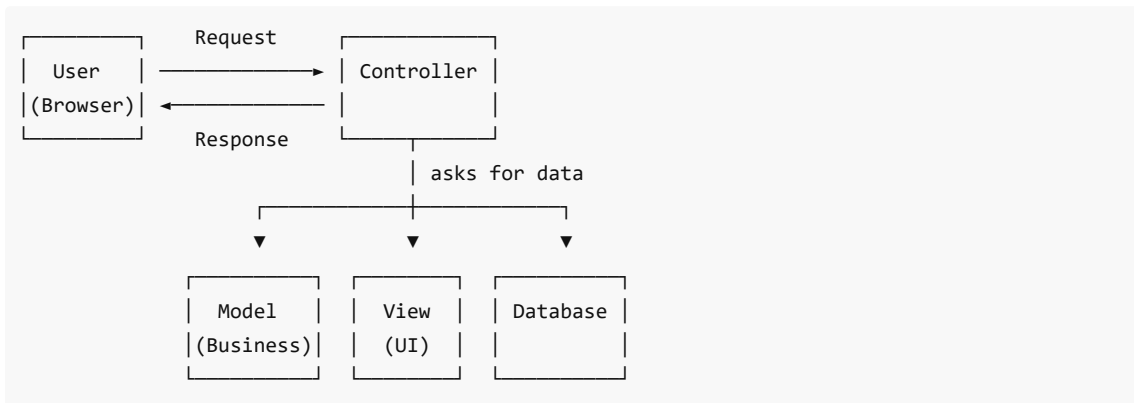
- Acts as the **middleman** between Model and View
- Controller = Request Handler + Decision Maker**
- Main responsibilities:
 - Handle User Requests** — receives HTTP requests (GET /users, POST /login)
 - Coordinate with Model** — calls Model methods to fetch/save/update/delete data
 - Return Response** — sends data to View (HTML) or as JSON (API-based)

MVC Flow (Step by Step)

User → Request → Controller → Model → Data → Controller → View → Response → User

1. User sends a request (e.g., GET /users)
2. **Controller** receives the request
3. Controller asks **Model** for data
4. Model queries database and returns data
5. Controller sends data to **View**
6. View renders and displays the result

MVC Diagram (Text-Based)



MTV Architecture (Django's Version)

Django uses a variation called **MTV (Model–Template–View)**:

Component	Role	MVC Equivalent
Model	Handles database structure and data	Model
Template	Handles UI (HTML response)	View
View	Handles business logic and request processing	Controller

Note: Django's "View" ≈ "Controller" in MVC. Django's "Template" ≈ "View" in MVC.

3.2 Role of Backend in Web Applications

Backend development refers to the server-side of a web application that works behind the scenes to process requests, manage data, apply business logic, and ensure security.

Major Features of Backend Development

#	Feature	Description
1	Request Handling & Routing	Receives HTTP requests (GET/POST/PUT/DELETE), routes to appropriate logic
2	Business Logic Implementation	Enforces rules (age validation, payment checks), controls workflows
3	Database Management	Stores, retrieves, updates, deletes data; ensures integrity

4	Authentication & Authorization	Authentication = verifies who user is; Authorization = controls what user can access
5	Security	Protects from SQL Injection, XSS, CSRF, brute-force attacks
6	API Development & Integration	Backend exposes APIs for frontend & mobile apps
7	Session & State Management	Manages user sessions using cookies, server-side sessions, token-based auth
8	Error Handling & Logging	Handles errors gracefully, logs for debugging
9	Performance & Scalability	Caching (Redis), load balancing, DB indexing, async processing
10	Data Validation & Sanitization	Ensures correct data format, prevents malicious input
11	File Handling	Upload and manage files, store images/documents/videos
12	Communication Services	Email notifications, SMS alerts, push notifications

Example: Online Shopping App

Backend handles:

- └─ User login (authentication)
- └─ Product storage (database)
- └─ Order processing (business logic)
- └─ Payment verification (external API)
- └─ Admin control (authorization)

Frontend: displays results only

Backend development is essential for handling business logic, database operations, authentication, security, APIs, and scalability.

3.3 Backend Web Framework: Django

Django is a high-level Python web framework used to build secure, scalable, and maintainable backend web applications quickly. It follows the **MVT (Model–View–Template)** architecture and promotes rapid development with clean and pragmatic design.

Core Features of Django

1. ORM (Object Relational Mapping)

- Interact with database using **Python classes** — no need to write SQL manually
- Supports MySQL, PostgreSQL, SQLite, Oracle

```
class Student(models.Model):
    name = models.CharField(max_length=100)
```

```
age = models.IntegerField()
email = models.EmailField(unique=True)
```

2. Built-in Admin Panel

- Automatic admin dashboard at `/admin`
- Manage database records visually
- Saves hours of development time

3. URL Routing System

- Maps URLs to views cleanly

```
# urls.py
urlpatterns = [
    path('students/', views.student_list),
    path('students/<int:id>', views.student_detail),
]
```

4. Authentication & Authorization

- Built-in system for: Login, Logout, Signup, Password hashing, User roles & permissions
- Secure by default

5. Form Handling

- Django forms handle: Validation, CSRF protection, Error handling

6. Security Features

Django protects against: SQL Injection, XSS, CSRF, Clickjacking

7. Template Engine

- Dynamic HTML rendering using template tags and filters

```
{% for student in students %}
    <p>{{ student.name }}</p>
{% endfor %}
```

8. REST API Support (DRF — Django REST Framework)

- Used to build API-only backends for React, Next.js, Mobile apps
- Features: Serialization, Authentication, Pagination, Permissions

9. Middleware Support

- Middleware sits between: Request → View → Response
- Used for: Authentication checks, Logging, Security, Session handling

Django Project File Structure

```
myproject/
├─ manage.py           # Command-line utility
├─ myproject/
│   └─ __init__.py
└─ settings.py         # Project configuration (DB, installed apps, middleware)
```

```

|   ├── urls.py          # Root URL configuration
|   ├── wsgi.py          # WSGI entry point (sync)
|   └── asgi.py          # ASGI entry point (async)
└── myapp/
    ├── __init__.py
    ├── admin.py         # Register models with admin
    ├── apps.py          # App configuration
    ├── models.py        # Database models
    ├── views.py         # View functions (controllers)
    ├── urls.py          # App-level URL configuration
    ├── forms.py         # Django forms
    ├── serializers.py   # DRF serializers
    ├── tests.py         # Test cases
    ├── migrations/      # Database migration files
    └── templates/       # HTML templates

```

Advantages of Django

Advantage	Description
Fast Development	Ready-made components, less boilerplate
Highly Secure	Security handled by framework
Scalable	From small apps to large platforms
Clean Code Structure	Easy to read and maintain
Large Community	Excellent documentation, lots of packages
Python Based	Easy for beginners, powerful for professionals

3.4 Handling Requests & Responses

Request–Response Cycle

Client → HTTP Request → Middleware → Route Handler (View) → Response → Client

- 1. Client (Browser/React/Mobile App) sends an HTTP request
- 2. Server processes the request
- 3. Server sends back an HTTP response

WSGI and ASGI

	WSGI	ASGI
Full Form	Web Server Gateway Interface	Asynchronous Server Gateway Interface
Type	Synchronous	Asynchronous
Use Case	Traditional web apps	Real-time apps (chat, live notifications)
Limitation	Handles one request at a time	Can handle multiple requests concurrently

Django Request Object

The `request` object contains all request data:

Attribute	Description
<code>request.method</code>	HTTP method (GET, POST, etc.)
<code>request.GET</code>	Query parameters from URL
<code>request.POST</code>	Form data from POST body
<code>request.body</code>	Raw request body
<code>request.headers</code>	Request headers
<code>request.user</code>	Currently logged-in user
<code>request.FILES</code>	Uploaded files

```
def user_view(request):
    user_id = request.GET.get('id')           # GET param
    name = request.POST.get('name')          # POST data
    user = request.user                       # logged-in user
    method = request.method                   # 'GET' or 'POST'
```

Django Response Classes

Response Type	Purpose	Example
<code>HttpResponse</code>	Plain text response	<code>HttpResponse("Hello")</code>
<code>JsonResponse</code>	JSON data response	<code>JsonResponse({"msg": "ok"})</code>
<code>HttpResponseRedirect</code>	Redirect to URL	<code>HttpResponseRedirect('/home/')</code>
<code>render()</code>	Render HTML template	<code>render(request, 'index.html', context)</code>

```
from django.http import HttpResponse, JsonResponse
from django.shortcuts import render, redirect

def hello(request):
    return HttpResponse("Hello from Django")

def api(request):
    return JsonResponse({"message": "Success", "status": 200})

def login_view(request):
    if request.method == "POST":
        return redirect('/dashboard/')
    return render(request, 'login.html')
```

Handling HTTP Methods in Django

```
def login(request):
    if request.method == "POST":
        username = request.POST.get('username')
        password = request.POST.get('password')
        # process login
        return HttpResponseRedirect("Login Success")
    elif request.method == "GET":
        return render(request, 'login.html')
    return HttpResponseRedirect("Method Not Allowed", status=405)
```

Node.js vs Django Comparison

Aspect	Node.js (Express)	Django
Language	JavaScript	Python
Architecture	Event-driven	MVT
Request Object	req	request
Response Object	res	HttpResponse
Routing	Inside app	Separate URL config
Middleware	Function-based	Class-based
Async Support	Native	Via ASGI
API Development	Lightweight	Structured

3.5 Form Data Handling & Sessions

Form Data Handling

Form data is the information sent by the client (browser) to the server when a user submits a form. Examples: login forms, registration forms, feedback forms, data entry forms.

Processing Form Data in Django

```
# views.py
from django.shortcuts import render, redirect
from django.contrib import messages

def register(request):
    if request.method == 'POST':
        name = request.POST.get('name', '').strip()
        email = request.POST.get('email', '').strip()
        password = request.POST.get('password', '')

        # Validation
```

```

    if not name or not email or not password:
        messages.error(request, 'All fields are required')
        return render(request, 'register.html')

    if len(password) < 8:
        messages.error(request, 'Password must be at least 8 characters')
        return render(request, 'register.html')

    # Save to database
    User.objects.create(name=name, email=email, password=password)
    return redirect('/login/')

return render(request, 'register.html')

```

CSRF Protection

CSRF (Cross-Site Request Forgery) protection in Django:

- Django automatically adds CSRF middleware
- Every POST form must include `{% csrf_token %}`

```

<form method="POST" action="/register/">
    {% csrf_token %}
    <input type="text" name="name" placeholder="Name">
    <input type="email" name="email" placeholder="Email">
    <button type="submit">Register</button>
</form>

```

Sessions in Django

A **session** is a server-side mechanism to store user-specific data across multiple requests.

How Sessions Work

1. User logs in
2. Server creates a session, stores data in DB/cache
3. Server sends **session ID** to client as a cookie
4. Client sends session ID with every request
5. Server looks up session data using the ID

Setting, Getting, and Deleting Sessions

```

# SETTING a session value
def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user:
            login(request, user)
            request.session['username'] = username    # Set session
            request.session['user_id'] = user.id      # Set session

```



```

        request.session.set_expiry(3600)                # Expires in 1 hour
        return redirect('/dashboard/')
    else:
        return HttpResponse("Invalid credentials")

# GETTING a session value
def dashboard(request):
    username = request.session.get('username', 'Guest') # Get session
    user_id = request.session.get('user_id')
    return render(request, 'dashboard.html', {'username': username})

# DELETING a session value
def logout_view(request):
    del request.session['username']                    # Delete specific key
    request.session.flush()                            # Delete entire session
    logout(request)
    return redirect('/login/')

```

Session Configuration in settings.py

```

# settings.py
SESSION_ENGINE = 'django.contrib.sessions.backends.db' # DB-backed sessions
SESSION_COOKIE_AGE = 3600                             # 1 hour in seconds
SESSION_EXPIRE_AT_BROWSER_CLOSE = True                 # Expire on browser close
SESSION_COOKIE_SECURE = True                           # HTTPS only
SESSION_COOKIE_HTTPONLY = True                         # No JS access

```

Cookies

A **cookie** is a small piece of data stored on the **client's browser**, sent by the server.

Setting, Getting, and Deleting Cookies

```

# SETTING a cookie
def set_cookie_view(request):
    response = HttpResponse("Cookie set!")
    response.set_cookie('username', 'sunil', max_age=3600) # expires in 1 hour
    return response

# GETTING a cookie
def get_cookie_view(request):
    username = request.COOKIE.get('username', 'Guest')
    return HttpResponse(f"Hello, {username}")

# DELETING a cookie
def delete_cookie_view(request):
    response = HttpResponse("Cookie deleted!")
    response.delete_cookie('username')
    return response

```

Sessions vs Cookies Comparison

Feature	Sessions	Cookies
Storage	Server-side	Client-side (browser)
Security	More secure	Less secure (exposed to client)
Data Size	Can store large data	Limited to ~4KB
Expiry	Server controls	Client/server can set
Dependency	Requires session ID cookie	Independent
Performance	Server load	Less server load
Use Case	Sensitive data (login state)	Preferences, tracking

3.6 Routing, Middleware, and Templates

Routing

Routing is the process of mapping a URL request to a view function that handles the request. In Django, routing is handled using **URL Configuration (URLconf)**.

Django URL Routing

```
# myproject/urls.py (Root URL config)
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('students/', include('students.urls')),    # Include app URLs
    path('api/', include('api.urls')),
]

# students/urls.py (App-level URL config)
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name='student-list'),
    path('<int:id>/', views.student_detail, name='student-detail'),
    path('create/', views.student_create, name='student-create'),
    path('<int:id>/update/', views.student_update, name='student-update'),
    path('<int:id>/delete/', views.student_delete, name='student-delete'),
]
```

URL Patterns

```
# Static URL
path('home/', views.home)

# Dynamic URL with integer parameter
path('student/<int:id>/', views.student_detail)

# Dynamic URL with string slug
path('post/<slug:slug>/', views.post_detail)

# URL with string
path('user/<str:username>/', views.user_profile)

# Regular expressions
re_path(r'^articles/(?P<year>[0-9]{4})/$', views.year_archive)
```

Middleware

Middleware is a component that processes requests before they reach the view and processes responses before they are sent back to the client. Middleware works **globally** on every request/response.

Middleware Position in Request/Response Cycle

```
Client Request
↓
[Middleware 1] ← e.g., SecurityMiddleware
↓
[Middleware 2] ← e.g., SessionMiddleware
↓
[Middleware 3] ← e.g., AuthenticationMiddleware
↓
View (request processing)
↓
[Middleware 3] ← response phase
↓
[Middleware 2]
↓
[Middleware 1]
↓
Client Response
```

Built-in Django Middleware

```
# settings.py
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
```

```
'django.middleware.clickjacking.XFrameOptionsMiddleware',  
]
```

Custom Middleware for Logging

```
# middleware.py  
import logging  
import time  
  
logger = logging.getLogger(__name__)  
  
class LoggingMiddleware:  
    def __init__(self, get_response):  
        self.get_response = get_response  
  
    def __call__(self, request):  
        # Before view (request phase)  
        start_time = time.time()  
        logger.info(f"REQUEST: {request.method} {request.path} from  
{request.META.get('REMOTE_ADDR')}")  
  
        # Call the view  
        response = self.get_response(request)  
  
        # After view (response phase)  
        duration = time.time() - start_time  
        logger.info(f"RESPONSE: {response.status_code} | Duration: {duration:.3f}s")  
  
        return response
```

Custom Middleware for Authentication Check

```
class AuthMiddleware:  
    def __init__(self, get_response):  
        self.get_response = get_response  
  
    def __call__(self, request):  
        protected_paths = ['/dashboard/', '/profile/', '/settings/']  
  
        if request.path in protected_paths and not request.user.is_authenticated:  
            from django.shortcuts import redirect  
            return redirect('/login/')  
  
        response = self.get_response(request)  
        return response
```

Register Custom Middleware

```
# settings.py
MIDDLEWARE = [
    ...
    'myapp.middleware.LoggingMiddleware',
    'myapp.middleware.AuthMiddleware',
]
```

Templates (Django Template Language — DTL)

Templating is the process of generating dynamic HTML pages by combining HTML with data from the view. Django uses the **Django Template Language (DTL)**.

Template Variables

```
<!-- Display a variable -->
<h1>Hello, {{ username }}!</h1>
<p>Your age is: {{ user.age }}</p>
```

Template Tags

```
<!-- If/else condition -->
{% if user.is_authenticated %}
    <p>Welcome, {{ user.username }}!</p>
{% else %}
    <a href="/login/">Login</a>
{% endif %}

<!-- For loop -->
<ul>
{% for student in students %}
    <li>{{ student.name }} - {{ student.email }}</li>
{% empty %}
    <li>No students found.</li>
{% endfor %}
</ul>

<!-- URL tag -->
<a href="{% url 'student-list' %}">View Students</a>

<!-- Static files -->
{% load static %}
<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Template Filters

```
{{ name|upper }}          <!-- UPPERCASE -->
{{ name|lower }}          <!-- lowercase -->
{{ text|truncatewords:10 }} <!-- First 10 words -->
{{ price|floatformat:2 }} <!-- 2 decimal places -->
```

```
{{ date|date:"Y-m-d" }}    <!-- Format date -->
{{ list|length }}          <!-- Length of list -->
```

Template Inheritance

```
<!-- base.html (Parent Template) -->
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Site{% endblock %}</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
    <nav>...</nav>

    {% block content %}
    <!-- Child content goes here -->
    {% endblock %}

    <footer>...</footer>
</body>
</html>

<!-- child.html -->
{% extends 'base.html' %}

{% block title %}Students List{% endblock %}

{% block content %}
    <h1>All Students</h1>
    {% for student in students %}
        <p>{{ student.name }}</p>
    {% endfor %}
{% endblock %}
```

Including Template Snippets

```
<!-- Include partial templates -->
{% include 'partials/navbar.html' %}
{% include 'partials/footer.html' %}

<!-- Include with context -->
{% include 'partials/card.html' with title="Hello" %}
```

Passing Context from View to Template

```
def student_list(request):
    students = Student.objects.all()
```

```
context = {
    'students': students,
    'title': 'Student List',
    'count': students.count(),
}
return render(request, 'students/list.html', context)
```

3.7 Overview & Comparison of Backend Frameworks

Django (Python)

Feature	Details
Language	Python
Architecture	MVT (Model-View-Template)
Type	Full-stack / Batteries-included
Strengths	Built-in admin, ORM, auth, security
Best For	Rapid development, content-heavy apps, APIs
Used By	Instagram, Pinterest, Disqus

Flask (Python)

Feature	Details
Language	Python
Architecture	No specific pattern (minimal)
Type	Micro-framework
Strengths	Lightweight, flexible, easy to learn
Best For	Small apps, microservices, prototypes
Weakness	No built-in ORM or admin

FastAPI (Python)

Feature	Details
Language	Python
Architecture	ASGI-based, async
Type	Modern, high-performance API framework
Strengths	Fastest Python framework, auto documentation (Swagger), async support, type hints
Best For	REST APIs, microservices, high-performance backends

.NET MVC (C#)

Feature	Details
Language	C#
Architecture	MVC
Type	Full-stack enterprise framework
Strengths	Enterprise-grade, strongly typed, high performance
Best For	Enterprise applications, Windows-based systems

Ruby on Rails (Ruby)

Feature	Details
Language	Ruby
Architecture	MVC
Type	Full-stack, convention over configuration
Strengths	Rapid development, elegant code, scaffolding
Best For	Startups, web apps with CRUD operations

Spring Boot (Java)

Feature	Details
Language	Java
Architecture	MVC/Layered
Type	Enterprise-grade, microservices-ready
Strengths	Highly scalable, secure, large ecosystem
Best For	Large-scale enterprise systems, microservices

Node.js + Express (JavaScript)

Feature	Details
Language	JavaScript
Architecture	Event-driven, non-blocking I/O
Type	Runtime + minimal framework
Strengths	Same language front+back, real-time, fast
Best For	Real-time apps (chat), APIs, streaming

Comprehensive Comparison Table

Feature	Django	Flask	FastAPI	.NET MVC	Rails	Spring Boot	Node.js
Language	Python	Python	Python	C#	Ruby	Java	JavaScript
Performance	Medium	Medium	Very High	High	Medium	Very High	High
Learning Curve	Medium	Easy	Easy	Hard	Easy	Hard	Easy
Built-in ORM	Yes	No	No	Yes	Yes	Yes	No
Admin Panel	Yes	No	No	Yes	Partial	No	No
Async Support	Via ASGI	Partial	Native	Yes	Partial	Yes	Native
Best For	Full apps	Microservices	APIs	Enterprise	Rapid dev	Enterprise	Real-time

3.8 Database Integration: Relational vs NoSQL, CRUD, ORM

Types of Databases

Relational Databases (SQL)

- Data stored in **tables with rows and columns**
- Uses **SQL (Structured Query Language)**
- Supports relationships (foreign keys, joins)
- Examples: **MySQL, PostgreSQL, SQLite, Oracle, SQL Server**
- Best for: structured data, complex queries, transactions

NoSQL Databases

- Data stored as **documents, key-value pairs, graphs, or columns**
- More flexible schema
- Examples: **MongoDB (document), Redis (key-value), Cassandra (column), Firebase**
- Best for: large-scale data, flexible schema, big data, real-time applications

SQL vs NoSQL Comparison

Feature	SQL (Relational)	NoSQL
Data Structure	Tables with fixed schema	Documents, key-value, graphs
Schema	Fixed/rigid	Flexible/dynamic
Scalability	Vertical	Horizontal
Transactions	ACID compliant	Eventual consistency
Query Language	SQL	Varies (MongoDB query, etc.)

Examples	MySQL, PostgreSQL	MongoDB, Redis, Firebase
Best For	Complex relationships	Big data, flexibility

CRUD Operations

CRUD = **C**reate, **R**ead, **U**ppdate, **D**eleate — the four basic database operations.

Operation	HTTP Method	SQL	Django ORM
Create	POST	INSERT	<code>.create()</code> / <code>.save()</code>
Read	GET	SELECT	<code>.all()</code> / <code>.get()</code> / <code>.filter()</code>
Update	PUT/PATCH	UPDATE	<code>.update()</code> / <code>.save()</code>
Delete	DELETE	DELETE	<code>.delete()</code>

ORM (Object Relational Mapping)

ORM is a technique that allows developers to interact with a database using an object-oriented language (like Python) instead of writing raw SQL.

Python Class	↔	Database Table
Object	↔	Row
Attribute	↔	Column

Benefits of ORM

1. **No SQL knowledge required** — use Python code instead
2. **Database independence** — switch databases without rewriting code
3. **Security** — prevents SQL injection automatically
4. **Maintainability** — cleaner, readable code
5. **Productivity** — less boilerplate code

Django ORM Examples

Defining Models

```
# models.py
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.IntegerField()
    gpa = models.FloatField(default=0.0)
    enrolled_date = models.DateField(auto_now_add=True)
    is_active = models.BooleanField(default=True)

    def __str__(self):
        return self.name
```

```
class Course(models.Model):
    title = models.CharField(max_length=200)
    students = models.ManyToManyField(Student, related_name='courses')
```

Common Django Model Field Types

Field Type	Usage
CharField(max_length=n)	Short text
TextField()	Long text
IntegerField()	Integer numbers
FloatField()	Decimal numbers
EmailField()	Email addresses
BooleanField()	True/False
DateField()	Date only
DateTimeField()	Date + Time
ForeignKey()	Many-to-one relationship
ManyToManyField()	Many-to-many relationship
OneToOneField()	One-to-one relationship
ImageField()	Image file upload
FileField()	Any file upload
URLField()	URL string
SlugField()	URL-friendly string

CRUD with Django ORM

```
# CREATE
student = Student.objects.create(name="Sunil", email="sunil@example.com", age=20)
# OR
student = Student(name="Sunil", email="sunil@example.com", age=20)
student.save()

# READ
all_students = Student.objects.all()           # Get all
student = Student.objects.get(id=1)            # Get by primary key
students = Student.objects.filter(age=20)      # Filter
students = Student.objects.filter(age__gte=18) # age >= 18
students = Student.objects.exclude(is_active=False)
student = Student.objects.first()
student = Student.objects.last()
count = Student.objects.count()
```

```

# UPDATE
student = Student.objects.get(id=1)
student.name = "Updated Name"
student.save()
# OR bulk update
Student.objects.filter(age=20).update(is_active=False)

# DELETE
student = Student.objects.get(id=1)
student.delete()
# OR bulk delete
Student.objects.filter(is_active=False).delete()

```

Database Migrations

```

# Create migration files
python manage.py makemigrations

# Apply migrations to database
python manage.py migrate

# Show all migrations
python manage.py showmigrations

```

3.9 Authentication & Authorization: Cookies, Sessions, JWT

Authentication vs Authorization

	Authentication	Authorization
Question	Who are you?	What can you do?
Process	Verify identity (username/password)	Check permissions/roles
Example	Login with credentials	Admin can delete, user can only view
Happens	First	After authentication

Login Flow in Django

```

from django.contrib.auth import authenticate, login, logout
from django.shortcuts import render, redirect

def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        # Authenticate user

```

```

        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)                # Creates session
            return redirect('/dashboard/')
        else:
            return render(request, 'login.html', {'error': 'Invalid credentials'})

    return render(request, 'login.html')

def logout_view(request):
    logout(request)                            # Destroys session
    return redirect('/login/')

```

JWT (JSON Web Token)

JWT is a stateless, token-based authentication method.

JWT Structure

Header.Payload.Signature

Example:

eyJhbGciOiJIUzI1NiJ9.eyJ1c2VybmFtZSI6InN1bm1s...LKJHasd

Part	Contains	Example
Header	Algorithm type, token type	{"alg": "HS256", "typ": "JWT"}
Payload	User data (claims)	{"user_id": 1, "username": "sunil", "exp": 1234567}
Signature	Hashed header+payload+secret	Ensures data integrity

JWT Authentication Flow

1. User sends: POST /login with {username, password}
2. Server validates credentials
3. Server generates JWT token
4. Server sends token to client
5. Client stores token (localStorage or cookie)
6. Client sends token in every request: Authorization: Bearer <token>
7. Server verifies token signature and extracts user info
8. Server processes request if token is valid

JWT in Django (using django-rest-framework-simplejwt)

```

# Installation
pip install django-rest-framework-simplejwt

# settings.py
REST_FRAMEWORK = {

```

```

'DEFAULT_AUTHENTICATION_CLASSES': (
    'rest_framework_simplejwt.authentication.JWTAuthentication',
),
},

# urls.py
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns = [
    path('api/token/', TokenObtainPairView.as_view()),      # Login → get token
    path('api/token/refresh/', TokenRefreshView.as_view()), # Refresh token
]

```

Session-based vs JWT-based Authentication

Feature	Session-based	JWT-based
Storage	Server-side	Client-side (token)
State	Stateful	Stateless
Scalability	Harder (shared sessions)	Easier (no server state)
Security	Session hijacking risk	Token theft risk
Expiry	Server controls	Token contains expiry
Best For	Traditional web apps	APIs, microservices, mobile apps

Password Hashing

```

from django.contrib.auth.hashers import make_password, check_password

# Hash a password
hashed = make_password("mypassword123")
# Result: pbkdf2_sha256$260000$salt$hash

# Verify password
is_valid = check_password("mypassword123", hashed) # Returns True/False

# Django does this automatically when creating users
user = User.objects.create_user(username='sunil', password='pass123')

```

3.10 Middleware for Logging, Error Handling, Security

Middleware for Logging

```

# middleware/logging.py
import logging
import time

```

```

import json

logger = logging.getLogger('django.request')

class RequestLoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start_time = time.time()

        # Log incoming request
        logger.info(f"[REQUEST] {request.method} {request.path} | "
                    f"IP: {request.META.get('REMOTE_ADDR')} | "
                    f"User: {request.user}")

        response = self.get_response(request)

        # Log outgoing response
        duration = (time.time() - start_time) * 1000
        logger.info(f"[RESPONSE] {response.status_code} | "
                    f"Duration: {duration:.2f}ms | Path: {request.path}")

        return response

```

Middleware for Error Handling

```

# middleware/error_handling.py
from django.http import JsonResponse
import traceback
import logging

logger = logging.getLogger(__name__)

class ErrorHandlingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        try:
            response = self.get_response(request)
            return response
        except Exception as e:
            # Log the full traceback
            logger.error(f"Unhandled Exception: {str(e)}\n{traceback.format_exc()}")

            # Return user-friendly error response
            return JsonResponse({
                "error": "An internal server error occurred",

```

```
        "status": 500
    }, status=500)
```

Middleware for Security

```
# middleware/security.py
class SecurityHeadersMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        # Add security headers to every response
        response['X-Content-Type-Options'] = 'nosniff'
        response['X-Frame-Options'] = 'DENY'
        response['X-XSS-Protection'] = '1; mode=block'
        response['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
        response['Content-Security-Policy'] = "default-src 'self'"

        return response
```

Registering All Custom Middlewares

```
# settings.py
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'myapp.middleware.logging.RequestLoggingMiddleware',      # Custom logging
    'myapp.middleware.error_handling.ErrorHandlingMiddleware', # Custom errors
    'myapp.middleware.security.SecurityHeadersMiddleware',    # Custom security
    'django.contrib.sessions.middleware.SessionMiddleware',
    ...
]
```

Django Logging Configuration

```
# settings.py
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'handlers': {
        'file': {
            'level': 'DEBUG',
            'class': 'logging.FileHandler',
            'filename': 'debug.log',
        },
        'console': {
            'level': 'INFO',
```



```
        'class': 'logging.StreamHandler',
    },
},
'loggers': {
    'django': {
        'handlers': ['file', 'console'],
        'level': 'INFO',
        'propagate': True,
    },
},
}
```

Model Exam Questions with Answers

Q: Explain MVC Architecture with neat diagram. [8]

Answer:

MVC (Model–View–Controller) is a software design pattern that separates an application into three interconnected components, making code organized, maintainable, and scalable.

Model:

- Represents the data layer and business logic
- Interacts directly with the database
- Performs CRUD (Create, Read, Update, Delete) operations
- Example: A `Student` class that maps to a `students` database table

View:

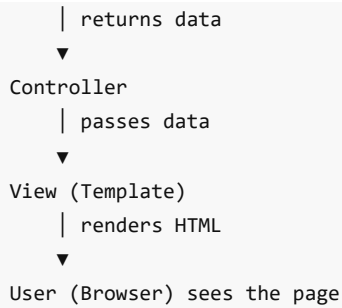
- Responsible for presentation — what the user sees
- Renders data received from the Controller
- No direct database access or business logic
- In server-rendered apps: HTML templates (Jinja2, EJS)
- In modern SPAs: React/Vue/Angular components

Controller:

- Acts as the middleman between Model and View
- Receives HTTP requests from the user
- Calls Model to fetch/manipulate data
- Passes data to View for rendering
- Returns the final response (HTML or JSON)

MVC Flow:

```
User (Browser)
  | sends HTTP request
  ▼
Controller
  | asks for data
  ▼
Model ↔ Database
```



Example in Django (MTV):

```
# Model (models.py)
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()

# View/Controller (views.py)
def product_list(request):
    products = Product.objects.all()          # Ask Model for data
    return render(request, 'list.html', {'products': products}) # Pass to View

# Template/View (list.html)
{% for product in products %}
    <p>{{ product.name }} - Rs. {{ product.price }}</p>
{% endfor %}
```

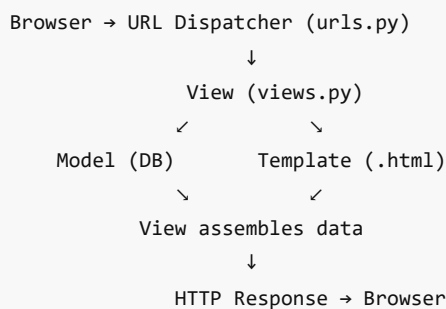
Q: Explain MTV Architecture with neat diagram. [8]

Answer:

Django uses **MTV (Model–Template–View)** which is a variation of MVC:

MTV Component	Equivalent	Role
Model	Model	Database structure & data logic
Template	View	UI rendering (HTML)
View	Controller	Business logic & request handling

Flow:



Code Example:

```
# Model
class Student(models.Model):
    name = models.CharField(max_length=100)

# View (like a Controller)
def student_list(request):
    students = Student.objects.all()
    return render(request, 'students.html', {'students': students})

# Template (like a View)
# students.html
{% for s in students %}<p>{{ s.name }}</p>{% endfor %}
```

Q: Explain how to set, get and delete sessions. [4]

Answer:

A session is a server-side storage mechanism that maintains user state across multiple HTTP requests.

```
# SETTING a session
def login_view(request):
    if request.method == 'POST':
        user = authenticate(request, username=request.POST['username'],
                             password=request.POST['password'])

        if user:
            login(request, user)
            request.session['user_id'] = user.id          # Set session key
            request.session['username'] = user.username
            request.session.set_expiry(3600)              # 1 hour expiry
            return redirect('/dashboard/')

# GETTING a session value
def dashboard(request):
    username = request.session.get('username', 'Guest') # Get with default
    user_id = request.session.get('user_id')
    return render(request, 'dashboard.html', {'username': username})

# DELETING a session
def logout_view(request):
    # Delete specific key
    if 'username' in request.session:
        del request.session['username']

    # OR delete entire session
    request.session.flush()
    logout(request)
    return redirect('/login/')
```

Q: Explain how to set, get and delete cookies from server side. [4]

Answer:

```
# SETTING a cookie
def set_cookie(request):
    response = HttpResponse("Cookie Set!")
    response.set_cookie(
        key='username',
        value='sunil',
        max_age=3600,          # Seconds (1 hour)
        httponly=True,        # Prevents JS access
        secure=True,          # HTTPS only
        samesite='Lax'        # CSRF protection
    )
    return response

# GETTING a cookie
def get_cookie(request):
    username = request.COOKIES.get('username', 'Guest')
    return HttpResponse(f"Hello {username}")

# DELETING a cookie
def delete_cookie(request):
    response = HttpResponse("Cookie Deleted!")
    response.delete_cookie('username')
    return response
```

Q: What is Django? Explain file structure of Django Project and Django App. [1+4]

Answer:

Django is a high-level Python web framework that enables rapid development of secure, scalable web applications. It follows the MVT (Model–View–Template) pattern and is known as a "batteries-included" framework because it comes with built-in tools for authentication, ORM, admin panel, form handling, and security.

Django Project Structure:

```
myproject/
├─ manage.py          # CLI utility (runserver, makemigrations, migrate)
├─ myproject/
│   ├─ __init__.py    # Makes it a Python package
│   ├─ settings.py    # All project configurations (DB, apps, middleware)
│   ├─ urls.py        # Root URL routing
│   ├─ wsgi.py        # For deployment (synchronous)
│   └─ asgi.py        # For async deployment
└─ myapp/
    ├─ __init__.py
    ├─ admin.py       # Register models for admin panel
    ├─ apps.py        # App configuration
    ├─ models.py      # Database models (classes → tables)
    └─ views.py       # Request handling logic
```

```
|— urls.py          # URL patterns for this app
|— forms.py         # Django form classes
|— tests.py         # Unit tests
|— serializers.py   # For DRF (API serialization)
|— migrations/      # Auto-generated migration files
    |— 0001_initial.py
templates/          # HTML templates
static/             # CSS, JS, images
```

Q: What is Routing? How do you handle routing in server side? Give examples. [1+4]

Answer:

Routing is the process of mapping incoming URL requests to specific view functions or controllers that handle those requests.

In Django, routing is managed through the `urls.py` file using URL patterns:

```
# myproject/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('home.urls')),
    path('students/', include('students.urls')),
    path('api/', include('api.urls')),
]

# students/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name='student-list'),          # /students/
    path('<int:id>/', views.student_detail, name='detail'),        # /students/5/
    path('create/', views.create_student),                      # /students/create/
    path('<int:id>/edit/', views.edit_student),                   # /students/5/edit/
    path('<int:id>/delete/', views.delete_student),               # /students/5/delete/
]

# views.py
def student_list(request):
    students = Student.objects.all()
    return render(request, 'students/list.html', {'students': students})

def student_detail(request, id):
    student = get_object_or_404(Student, id=id)
    return render(request, 'students/detail.html', {'student': student})
```

Q: What is Templating? How do you generate dynamic HTML content in server side? [1+4]

Answer:

Templating is the process of generating dynamic HTML pages by embedding data and logic inside HTML templates. Django uses the **Django Template Language (DTL)**.

```
# views.py - pass data to template
def home(request):
    context = {
        'title': 'Student Portal',
        'students': Student.objects.all(),
        'count': Student.objects.count(),
    }
    return render(request, 'home.html', context)
```

```
<!-- home.html - dynamic HTML -->
<!DOCTYPE html>
<html>
<head><title>{{ title }}</title></head>
<body>
    <h1>Total Students: {{ count }}</h1>

    {% if students %}
        <table>
        {% for student in students %}
            <tr>
                <td>{{ forloop.counter }}</td>
                <td>{{ student.name }}</td>
                <td>{{ student.email }}</td>
            </tr>
        {% endfor %}
        </table>
    {% else %}
        <p>No students enrolled yet.</p>
    {% endif %}
</body>
</html>
```

Q: What are static files? What is ORM? What are benefits of ORM? [2+2+2]

Answer:

Static Files: Files that are served as-is to the browser without any processing — CSS stylesheets, JavaScript files, images, fonts. In Django:

```
# settings.py
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

```
# In template
{% load static %}
<link href="{% static 'css/style.css' %}" rel="stylesheet">
```

ORM (Object Relational Mapping): A programming technique that maps database tables to Python classes (or objects), allowing developers to interact with the database using Python code instead of raw SQL.

Python Class	↔	Database Table
Object	↔	Row
Attribute	↔	Column

Benefits of ORM:

1. **No Raw SQL** — use Python syntax instead
2. **Database Independence** — switch MySQL to PostgreSQL without rewriting code
3. **Security** — prevents SQL injection automatically
4. **Readability** — cleaner, more maintainable code
5. **Productivity** — faster development with less boilerplate

Q: What is middleware? Create custom middleware for logging. [2+4]

Answer:

Middleware is a component that sits between the request and response cycle. It processes every request before it reaches the view and every response before it is sent to the client. It works **globally** for all requests.

Uses of middleware: Authentication checks, CSRF protection, Logging, Session handling, Security headers, Rate limiting.

Custom Logging Middleware:

```
# middleware/logging.py
import logging
import time

logger = logging.getLogger(__name__)

class RequestLoggingMiddleware:
    """Logs every request and response with timestamp and duration."""

    def __init__(self, get_response):
        self.get_response = get_response  # Next middleware or view

    def __call__(self, request):
        # ---- BEFORE VIEW ----
        start = time.time()
        logger.info(
            f"[{start}] REQUEST: {request.method} {request.path} "
            f"| IP: {request.META.get('REMOTE_ADDR', 'unknown')} "
            f"| User: {getattr(request, 'user', 'anonymous')}"
        )
```

```

        # Call next middleware or view
        response = self.get_response(request)

        # ---- AFTER VIEW ----
        duration = round((time.time() - start) * 1000, 2)
        logger.info(
            f"RESPONSE: {response.status_code} "
            f"| Path: {request.path} "
            f"| Duration: {duration}ms"
        )

        return response

# Register in settings.py
# MIDDLEWARE = [..., 'myapp.middleware.logging.RequestLoggingMiddleware']

```

Q: Explain with example syntax of Django Template Language. [6]

Answer:

Django Template Language (DTL) allows embedding Python-like logic inside HTML templates.

```

<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}Default Title{% endblock %}</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>

<!-- 1. VARIABLES: {{ variable }} -->
<h1>Welcome, {{ username }}!</h1>
<p>Your GPA: {{ student.gpa|floatformat:2 }}</p>

<!-- 2. FILTERS: {{ value|filter }} -->
<p>{{ name|upper }}</p>           <!-- SUNIL -->
<p>{{ bio|truncatewords:20 }}</p> <!-- First 20 words... -->
<p>{{ date|date:"F j, Y" }}</p>   <!-- March 18, 2026 -->

<!-- 3. IF STATEMENTS: {% if %} -->
{% if student.gpa >= 3.5 %}
    <p class="honor">Honor Student</p>
{% elif student.gpa >= 2.5 %}
    <p>Average Student</p>
{% else %}
    <p>Needs Improvement</p>
{% endif %}

<!-- 4. FOR LOOPS: {% for %} -->
<ul>

```



```

{% for course in courses %}
    <li>{{ forloop.counter }}. {{ course.title }}</li>
{% empty %}
    <li>No courses enrolled.</li>
{% endfor %}
</ul>

<!-- 5. URL TAG: {% url %} -->
<a href="{% url 'student-list' %}">All Students</a>
<a href="{% url 'student-detail' id=student.id %}">View</a>

<!-- 6. TEMPLATE INHERITANCE -->
{% extends 'base.html' %}
{% block content %}
    <!-- page-specific content -->
{% endblock %}

<!-- 7. INCLUDE PARTIAL -->
{% include 'partials/navbar.html' %}

<!-- 8. CSRF TOKEN for forms -->
<form method="POST">
    {% csrf_token %}
    <!-- form fields -->
</form>

<!-- 9. COMMENTS -->
{# This is a comment, not visible in output #}

</body>
</html>

```

Q: What are different field types for defining models in Django? Give example. [6]

Answer:

```

from django.db import models

class Patient(models.Model):
    # Text Fields
    name = models.CharField(max_length=100)           # Short text (required max_length)
    address = models.TextField()                      # Long text, unlimited
    slug = models.SlugField(unique=True)               # URL-friendly: "john-doe-123"

    # Numeric Fields
    age = models.IntegerField()                       # Integer
    weight = models.FloatField()                      # Floating point
    fee = models.DecimalField(max_digits=10, decimal_places=2) # Precise decimal

    # Boolean
    is_admitted = models.BooleanField(default=False)

```

```

# Date/Time
dob = models.DateField() # Date only: 2000-03-18
admitted_at = models.DateTimeField(auto_now_add=True) # Auto on creation
updated_at = models.DateTimeField(auto_now=True) # Auto on each save

# Contact
email = models.EmailField(unique=True)
mobile = models.CharField(max_length=15)
website = models.URLField(blank=True)

# Files
photo = models.ImageField(upload_to='photos/') # Image upload
report = models.FileField(upload_to='reports/') # Any file upload

# Relationships
doctor = models.ForeignKey('Doctor', on_delete=models.CASCADE) # Many-to-one

# Other
gender = models.CharField(max_length=10, choices=[('M', 'Male'), ('F', 'Female')])

def __str__(self):
    return self.name

```

Q: What is Database Migration? What are the major differences between Sessions and Cookies? [2+4]

Answer:

Database Migration: The process of propagating changes made to Django models (Python classes) to the actual database schema. When you add/modify/delete a model field, you need to create and apply migrations.

```

python manage.py makemigrations # Detects model changes, creates migration file
python manage.py migrate        # Applies migration to database
python manage.py showmigrations # Shows all migration status

```

Sessions vs Cookies:

Feature	Sessions	Cookies
Storage Location	Server-side (DB/memory)	Client-side (browser)
Security	More secure (data not exposed)	Less secure (stored in browser)
Data Size	Can store large data	Max ~4KB per cookie
Access	Only accessible server-side	Accessible by JS (unless HttpOnly)
Expiry	Server controls expiry	Set via max_age/expires
Performance	Requires DB lookup	No server lookup needed

Use Case	Login state, sensitive data	Preferences, tracking, tokens
Dependency	Needs a session ID cookie	Independent

Q: Compare Django, Flask, FastAPI, ASP.NET, Spring Boot, and Node.js. [4]

Answer:

Feature	Django	Flask	FastAPI	ASP.NET	Spring Boot	Node.js
Language	Python	Python	Python	C#	Java	JavaScript
Type	Full-stack	Micro	Modern API	Enterprise	Enterprise	Runtime
ORM Built-in	Yes	No	No	Yes	Yes	No
Performance	Medium	Medium	Very High	High	Very High	High
Async	Via ASGI	Partial	Native	Yes	Yes	Native
Admin Panel	Yes	No	Auto Swagger	Yes	No	No
Learning Curve	Medium	Easy	Easy	Hard	Hard	Easy
Best For	Full apps, CMS	Microservices	APIs	Enterprise	Enterprise	Real-time

Q: Write a controller (view) that checks login credentials [7]

Answer:

```
# models.py
from django.db import models

class Student(models.Model):
    username = models.CharField(max_length=50, unique=True)
    password = models.CharField(max_length=100)
    name = models.CharField(max_length=100)

# views.py
from django.shortcuts import render, redirect
from django.contrib import messages
from .models import Student

def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username', '').strip()
        password = request.POST.get('password', '').strip()

        # Validate inputs
```

```

    if not username or not password:
        messages.error(request, 'Username and password are required')
        return render(request, 'login.html')

    # Check credentials against Student table
    try:
        student = Student.objects.get(username=username, password=password)
        # Credentials match - set session and redirect
        request.session['student_id'] = student.id
        request.session['student_name'] = student.name
        return redirect('/dashboard/')
    except Student.DoesNotExist:
        # Credentials don't match
        messages.error(request, 'Invalid username/password')
        return render(request, 'login.html')

    return render(request, 'login.html')

def dashboard(request):
    if not request.session.get('student_id'):
        return redirect('/login/')
    name = request.session.get('student_name', 'Student')
    return render(request, 'dashboard.html', {'name': name})

# login.html
"""
<form method="POST" action="/login/">
    {% csrf_token %}
    <input type="text" name="username" placeholder="Username" required>
    <input type="password" name="password" placeholder="Password" required>
    <button type="submit">Login</button>
    {% if messages %}
        {% for message in messages %}
            <p style="color:red;">{{ message }}</p>
        {% endfor %}
    {% endif %}
</form>
"""

```

Q: Write server side code to upload a file and validate file extension and size. [6]

Answer:

```

# views.py
from django.shortcuts import render
from django.http import HttpResponse
import os

def upload_file(request):
    if request.method == 'POST':
        uploaded_file = request.FILES.get('file')

```

```

if not uploaded_file:
    return render(request, 'upload.html', {'error': 'No file uploaded'})

# 1. Validate file extension
allowed_extensions = ['jpg', 'jpeg', 'png', 'gif']
file_name = uploaded_file.name
file_ext = file_name.rsplit('.', 1)[-1].lower() if '.' in file_name else ''

if file_ext not in allowed_extensions:
    return render(request, 'upload.html', {
        'error': f'Invalid file type. Allowed: {", ".join(allowed_extensions)}'
    })

# 2. Validate file size (max 2MB = 2 * 1024 * 1024 bytes)
max_size = 2 * 1024 * 1024
if uploaded_file.size > max_size:
    return render(request, 'upload.html', {'error': 'File size must be less than
2MB'})

# Save the file
save_path = os.path.join('media/uploads', file_name)
with open(save_path, 'wb+') as destination:
    for chunk in uploaded_file.chunks():
        destination.write(chunk)

return render(request, 'upload.html', {'success': 'File uploaded successfully!'})

return render(request, 'upload.html')

```

```

<!-- upload.html -->
<form method="POST" enctype="multipart/form-data">
    {% csrf_token %}
    <input type="file" name="file" accept=".jpg,.jpeg,.png,.gif">
    <button type="submit">Upload</button>
    {% if error %}<p style="color:red;">{{ error }}</p>{% endif %}
    {% if success %}<p style="color:green;">{{ success }}</p>{% endif %}
</form>

```